

Chapter 1: What is Retrieval-Augmented Generation

Retrieval-Augmented Generation, commonly abbreviated as RAG, is an architecture that combines a retrieval system with a generative language model. Instead of relying solely on the knowledge baked into a model's parameters during training, a RAG system looks up relevant passages from an external knowledge source at query time.

This approach helps address two common limitations of large language models: outdated knowledge and hallucination. By grounding the model's output in retrieved documents, the generated answer can be more accurate and verifiable.

A typical RAG pipeline consists of three stages: indexing, retrieval, and generation. Indexing involves splitting documents into chunks and converting them into vector embeddings. Retrieval involves finding the most relevant chunks for a given query. Generation involves feeding those chunks, along with the query, into a language model to produce a final answer.

Chapter 2: Document Loading and Splitting

Before any retrieval can happen, raw documents must be loaded and transformed into a structured format. Document loaders handle the parsing of various file types such as PDFs, Word documents, HTML pages, and plain text files.

Once loaded, long documents are typically split into smaller chunks. This is necessary because embedding models and language models have limited context windows, and smaller chunks tend to produce more precise retrieval results.

Common splitting strategies include fixed-size character splitting, recursive splitting that respects paragraph and sentence boundaries, and semantic splitting that groups text based on meaning. The choice of chunk size and overlap can significantly affect retrieval quality.

Chapter 3: Embeddings and Vector Stores

An embedding model converts text into a numerical vector that captures its semantic meaning. Texts with similar meaning end up with vectors that are close together in the embedding space, typically measured using cosine similarity or dot product.

Vector stores are specialized databases designed to store these embeddings and perform fast similarity search. Popular options include FAISS for local, in-memory search, as well as hosted solutions such as Pinecone, Weaviate, and Chroma.

When a query comes in, it is embedded using the same embedding model, and the vector store returns the chunks whose embeddings are most similar to the query embedding. This set of retrieved chunks forms the context that will be passed to the language model.

Chapter 4: Generation and Prompting

The final stage of a RAG pipeline combines the retrieved context with the original user query inside a prompt template. This prompt instructs the language model to answer the question using only the provided context, which helps reduce hallucination.

A well-designed prompt will also instruct the model on how to behave when the retrieved context does not contain enough information to answer the question, for example by asking it to say it does not know rather than guessing.

After generation, some RAG systems also return the source documents alongside the answer, allowing users to verify the response against the original material. This traceability is one of the key advantages of RAG over pure parametric generation.

Chapter 5: Evaluating a RAG System

Evaluating RAG systems requires looking at both the retrieval and generation components separately. Retrieval metrics include precision and recall of the retrieved chunks relative to a labeled set of relevant passages.

Generation quality can be evaluated using metrics such as faithfulness, which measures whether the answer is actually supported by the retrieved context, and relevance, which measures whether the answer addresses the user's question.

Many practitioners also use LLM-based evaluation, where a separate language model judges the quality of generated answers against a rubric. This can scale better than human evaluation, though it introduces its own biases and should be validated periodically.